A Uniform-Premise Presentation of iPRA

A persistent truth and a successful-trace invariant remove a branch from the complete proof checker

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We give a new proof presentation of the inherited scalar-iteration axiomatization of full first-order intuitionistic primitive recursive arithmetic. Every instruction now verifies a major premise. Instructions that previously had no major use a persistent, proved truth. A simultaneous induction on successful checking traces proves that every goal and every context entry is array rooted. Consequently the checker itself excludes scalar major expressions, including a false sentinel that would otherwise permit an unsound equality rewrite. No extra input validator or trusted well-formedness promise is required.

The clearer complete Python checker has 548 normalized bytes, 262 significant tokens and 303 syntax-tree nodes. A compact profile with the same canonical consequence relation has 537 bytes, 263 tokens and 308 nodes. Their maximum-compression gzip sizes are respectively 347 and 342 bytes. The current supplied collection has coordinatewise minima of 558 bytes, 262 tokens and 296 nodes, attained by different programs. Thus both delivered checkers beat its source-byte minimum, but neither is strictly smaller in every reported metric.

Canonical numerical and formula syntax, logical strength, and the whole-domain bi-interpretation with iPRA are preserved. The report states the entire primitive theory, proves the new trace and translation theorems, and reproduces the attributed arithmetic and logical bridge. The archive supplies executable certificates, adversarial controls, source measurements and all 443 distinct Python sources in the pinned comparison corpus. This establishes a structural reduction of the proof presentation. It does not establish an absolute simplest axiomatization or the requested major new arithmetic foundation.

Established: a complete 548-byte straightforward checker, or 537-byte compact checker, below the current 558-byte corpus minimum; the same canonical native consequence relation; full iPRA bi-interpretability and equiconsistency by the supplied written proofs.

Not established: absolute minimality, simultaneous dominance of all size metrics, a new arithmetic foundation, or Lean/Rocq verification.

Contents

1 The result and its precise scope **3**

2 The entire native theory **4**

3 Why the theory is full intuitionistic PRA **5**

4 The complete implementation and its input language **7**

5 Opening, arithmetic generation and the exact scope boundary **9**

6 A persistent truth removes the optional-major branch **11**

7 Validation, counterchecks and reproducibility **15**

8 Assessment of the requested breakthrough **16**

Attribution for the complete inherited bridge **17**

A Complementary introductions with one local assumption **17**

B Arithmetic strength and both ordinary interpretation composites **22**

C Internal scalar predecessor bootstrap **24**

D From literal-equation induction to full intuitionistic PRA **28**

E Pairing, primitive recursion, and internal numerical composites **34**

1 The result and its precise scope

The useful simplification is to make a premise compulsory. The old checker first asks whether an instruction has a major premise. The new checker always verifies its major; an instruction without substantive premises names the empty-array truth already present in the context. This removes a proof-control branch, rather than omitting the obligation to validate a submitted major.

There is a real correctness issue here. The implementation uses Python's false value as one nonmatching sentinel in its combined equality and closure test. Numeric zero compares equal to that sentinel. Simply deleting the old guard, or silently trusting a list-typed input interface, would therefore leave a soundness obligation. The new successful-trace invariant discharges it: no scalar goal can have a successful proof from the array-rooted contexts generated by this checker. The recursive proof check is itself enough to validate the relevant outer constructor.

1.1 What is new and what is inherited

The new contribution is the persistent neutral major, the relocation of the case/progress marker, and the mutual trace invariant that validates the resulting unguarded control flow. The complete native consequence relation is unchanged. Scalar iteration, complementary paired introductions, the Eq/Q/D logical encoding, the arity opener, constructed computation assumptions, and the nonempty certified zero are inherited from the supplied reports [2, 3, 4].

The comparison covers the current collection at the immutable commit

`76a2651601d17b34d46753f189ea026a21ce8d29`.

There are 17 immediate draft archives and 32 distinct recursive ZIP byte strings. The source census contains 443 distinct Python sources, with 2,239 occurrences after recursive archive deduplication. Of the 38 sources that meet at least one current minimum, three are complete standalone checkers and 35 are build, measurement, producer or test utilities. The complete source strings, origins, hashes and classification are included for offline remeasurement.

A separate search over 867 distinct textual sources found no previous unconditional-major implementation or checker initialized with the neutral truth context. Administrative true assumptions and shared continuations already exist in the collection and are credited. The novelty assertion is relative to this inspected corpus, not to the entire proof-theory literature.

1.2 The complexity comparison

Complete program	Bytes	Tokens	AST	Gzip-9
Old byte minimum	558	264	310	345
Old token/AST minimum	598	262	296	350
Old other AST minimum	689	266	296	369
Old explicit-list checker	566	264	308	351
New recommended checker	548	262	303	347
New compact checker	537	263	308	342

All source is charged: imports, input parsing, opening and scope checks, arithmetic, logical inference, and acceptance output. The producer is not imported by any checker. The Python interpreter and its standard JSON implementation are treated as the common platform in every row. The source normalizer removes comments and genuine docstrings, applies `ast.unparse`, and appends

a newline. Token counts exclude layout and comment tokens; AST counts include operator and context nodes. Gzip uses `gzip.compress(raw, compresslevel=9, mtime=0)` and is informational, not an optimization objective.

The recommended checker saves 10 normalized bytes against the smallest supplied byte profile (1.79 percent), and 18 bytes against the supplied explicit-list checker (3.18 percent). Its token count ties the supplied minimum; its AST count exceeds the minimum by seven. The compact profile saves 21 bytes against the byte minimum (3.76 percent), but has one more token and twelve more AST nodes than the separate respective minima. These are small, reproducible gains. The observed corpus minima are upper bounds on an unknown optimum, not mathematical lower-bound theorems.

2 The entire native theory

Here iPRA means full first-order intuitionistic primitive recursive arithmetic, with its primitive-recursive function symbols, defining equations, successor boundary laws, and quantifier-free induction. Atomic induction is an equivalent convention in the expanded PR language; Salehi uses the latter convention [6]. Our scalar language requires an internal equality-reflection argument before making that conversion; the argument is included, not assumed.

2.1 Numerical syntax and all nonlogical axioms

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only the state variable v in h is bound. All other parameters are fixed during iteration. There are exactly two computation schemas:

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

Substitution is capture free. These are staged finite descriptions: for each fixed earlier term $h(v, \vec{p})$, an ordinary first-order elaboration introduces $i_h(n, a, \vec{p})$ with those equations. No numerical variable quantifies over functions, and no total universal evaluator for primitive-recursive codes is postulated.

2.2 Formula syntax and every primitive logical rule

The native formulas are equations and a paired binder,

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A.$$

For convenient certificate contexts, also permit a closure $\mathsf{Q}_y B$. It is eliminable in favor of $\mathsf{D}_{u,y}B$, with u unused. Its intended ordinary meanings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Contexts Γ are finite reusable lists of formulas. There are the two computation axiom schemas above and precisely the following logical inference patterns:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{Hyp, Tr, QE}$$

Here F is any fixed native formula template, including quantified ones. Equality transport is not restricted to atomic formulas. Paired elimination is

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \quad \Gamma, \mathsf{Q}_y B(a,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter a is fresh for Γ, A and the fixed parameters of B. Paired introduction has two alternatives:

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}E(x,y)}. \tag{DI-case, DI-progress}$$

The progress alternative requires a literal numerical equation E. An equation beneath D or Q does not qualify. In each alternative, z is fresh for the retained context, witness and fixed parameters. The displayed local hypothesis is discharged.

This is the entire primitive theory $\mathcal{A}$. No separate intuitionistic connective axiom, reflexivity axiom, successor-injectivity axiom, coverage axiom, falsity rule, or further induction schema is primitive. The native contradiction is $0 = S0$. Weakening, exchange, contraction, renaming, numerical substitution and ordinary-assumption cut are admissible by induction on finite derivations, with the usual renaming of eigenparameters before insertion.

2.3 Derived administrative rules used by the checker

A computation axiom can equivalently be supplied as a local hypothesis:

$$\frac{\Gamma, E \vdash A}{\Gamma \vdash A} \quad (E \text{ an I0 or IS instance}). \tag{Supply}$$

The equivalence is Hyp in one direction and cut with the computation theorem in the other. Since $0 = 0$ is derived, it may be retained in every context. A second admissible rule handles raw computation data:

$$\vdash H(\vec{0}), \qquad \Gamma, H(\vec{b}) \vdash A \implies \Gamma \vdash A, \qquad \vec{b} \notin \mathrm{FV}(\Gamma, A). \tag{Zero instance}$$

Substitute zero for the fresh names in the second proof and cut in the first. This rule is an ordinary finite proof transformation. It is not a consistency principle, an oracle, or an assumption that $H(\vec{b})$ is universally true.

3 Why the theory is full intuitionistic PRA

The full inherited derivations appear in the attributed appendices. The essential constructions and the interpretation composites are recorded here so the exact claim is visible in the main report.

3.1 Recovering all intuitionistic connectives

Put $1 = S0$, and define

$$C(v,a,b) = I(v,a;u.b), \qquad \delta(v) = C(v,1,0), \qquad p(z) = I(z,0;u.\delta(u)).$$

Then $C(0,a,b) = a$, $C(Sv,a,b) = b$, $p(1) = 1$, and $p(2) = 0$. On equations define

$$G_v(s = t) = (C(v,0,s) = C(v,0,t)), \quad M_v(s = t, r = q) = (C(v,s,r) = C(v,t,q)).$$

Extend these operations through paired binders. Different binder depths are aligned using the derived padding equivalence $A \dashv\vdash \mathsf{D}_{u,y} G_y A$, with u unused. Fixed formula syntax recursion gives

$$\vdash G_0 A, \quad G_{Sv} A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \quad M_{Sv}(A,B) \dashv\vdash B.$$

The full logical interface is therefore expressible by native formulas:

$$\begin{aligned} A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A,B), \\ A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &= (0 = 1). \end{aligned}$$

All newly introduced variables are fresh. The zero conjunct in the universal macro is essential. Conjunction projections use instances one and two. Case introduction, DE and QE prove the ordinary introduction and elimination rules for these macros. Explosion from $0 = 1$ gives every equation by transport through C, then every native formula by external induction on its finite syntax.

The unused-witness progress rule gives contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

Case introduction gives numerical cases for arbitrary native formulas. Thus this is full intuitionistic logic, including disjunction and existential quantification, not merely its equational or negative part. No excluded-middle rule for arbitrary quantified formulas has been added.

3.2 Arithmetic reconstruction without circularity

In ordinary intuitionistic first-order logic, let T_I have the staged iterator equations, contextual equation induction, and

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

This explicit falsity bridge is necessary in the ordinary realization: positive equations alone admit a singleton model. The native logical reconstruction interprets these principles in $\mathcal{A}$.

The arithmetic dependency order is strict. First, scalar iteration and equation induction construct predecessor P, with $P0 = 0$ and $P(Sx) = x$, using a factor map, an internally proved comparison recurrence, and saturated iteration. Pairing and a faithful equality detector are not assumed at this stage. Next, predecessor gives monus and a characteristic of actual equality on the whole domain. This converts each fixed quantifier-free formula into a characteristic equation and recovers quantifier-free induction. Only then are pairing π and projections L, H constructed.

For a fixed primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}), \vec{p}),$$

the scalar compiler is

$$\begin{aligned} V(n,\vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \\ f^*(n,\vec{p}) &= H(V(n,\vec{p})). \end{aligned}$$

Internal equation induction proves $L(V(n,\vec{p})) = n$ and the desired defining equations. Passive parameters remain fixed during each induction; diagonal substitution occurs only in a previously proved universally parameterized identity.

Theorem 3.1 (Whole-domain interpretation and both round trips)**.** *The ordinary realization of $\mathcal{A}$ and full first-order* iPRA *are bi-interpretable and equiconsistent. Both domains are the whole numerical domain, equality is actual equality, and both definable composite isomorphisms have graph $x = y$.*

Proof. Interpret each fixed scalar iterator in iPRA by primitive recursion whose update ignores the counter. Interpret every fixed PR description in the opposite direction with the scalar compiler above. The internal predecessor, equality-reflection and characteristic proofs preserve the successor axioms and every quantifier-free induction instance.

Equation induction proves uniqueness of each fixed recurrence with parameters held fixed. By external induction on a finite description, its double translation satisfies its original defining equations and is therefore internally equal to the original operation at every argument. The identity relation is internally total, bijective and preserves all interpreted operations. This is a statement about all internal elements, not just standard numerals.

The logical macros have their ordinary meanings. In the native theory,

$$\mathsf{Q}_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y B(y),$$
$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

In the forward paired direction, DE exposes the conditional closure; under its base, QE gives every instance, and the derived logical introductions apply. In the reverse direction, eliminate the existential and use case introduction under its base. Structural induction on formula syntax gives both formula composites. Falsity is preserved in both directions by the displayed bridge. Effective finite proof translations give equiconsistency; no self-consistency theorem is asserted. □

4 The complete implementation and its input language

The following is the entire recommended checker. The explanatory `code/readable_checker.py` is an expansion of it, not a dependency.

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <=
     a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g)
    if m in ([s, u], not s and [t] * 3) and i(t, u) == a:
        a = i(t, s)
    h = m * 3 or ([[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else (a := i(*a, t, 2)))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)])
q = json.load(open(0)).pop
c(q(), [[]])
print('ACCEPT')
```

Input is one JSON array. Its final element is the requested closed formula; earlier four-field packets are consumed from the end. An unused prefix has no effect. Acceptance means both exit status zero and exactly `ACCEPT` followed by a newline. Exceptions reject. No trusted `assert` is disabled by Python's optimization mode.

4.1 Sorted syntax and opening

The canonical encodings are

$$\mathsf{D}_{x,y}B \mapsto [B], \qquad s = t \mapsto [s,t], \qquad \mathsf{Q}_y B \mapsto [B,B,B].$$

Numerical and formula interpretations are sorted. Their array meanings are as follows; ternary formulas use their last field as the matrix.

Arity	Numerical meaning	Formula meaning
0	zero	$0 = 0$
1	successor	paired binder D
2	zero annotation	equation
3	initialized scalar iterator	conditional closure Q
at least 4	zero	$0 = 0$

Scalar padding at formula positions may retain the inherited truth reading. An optional falsity reading is also sound, but unnecessary for the reduction. Scalar expressions are never successful goals in the recommended checker; they are not added to its canonical object language.

At numerical root depth three, coordinates $0, 1, 2$ denote zero. Under protected slots, $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot. Admitted negative labels name rigid parameters. Formula root depth is $2p$, and numerical root depth is $3 + 2p$, under p protected slots. The child-depth increment is uniformly

$$d \longmapsto d + 5 - \text{arity}\,.$$

A unary numerical successor carries two dummy zero slots. The counter and initializer of an iterator carry one dummy zero slot; its update uses that slot for the genuine previous state. A paired formula has slots y, x, and a closure has slot y. The named producer performs the corresponding shifts in canonical encodings.

The opener preserves a scalar in $k \leq a < d$. At a numerical position, an escaping scalar is a substitution hole whose image is validated at numerical root depth three. At a formula position, escaping scalar padding becomes zero. Array annotations are traversed even when they are semantically ignored; a successful traversal preserves their constant meaning. Failed validation rejects rather than silently assuming that raw data have the canonical shape.

4.2 Instruction meaning

Every packet is $[m, s, u, t]$, and every packet first proves m. The canonical instructions are:

Kind	Fields and effect
Elimination	Nonempty m is the actual equality, closure or paired major. A matching endpoint rewrites the goal; its export is $m * 3$.
Computation	$m = []$, u a nonempty initializer. Falsy s selects zero computation; otherwise s is a successor predecessor. The update is t.
Case introduction	$m = []$, $u = []$; t is the witness. Retain the zero instance of the opened matrix.
Progress introduction	$m = []$, $u = 0$; t is the witness. Retain the arbitrary-index instance only for a literal equation.

In introductions, s is unused. Ordinary literal-zero witnesses are permitted. The persistent empty-array premise closes by Hyp and consumes no instruction. The proof-format change need not shorten serialized certificates or improve checking speed; it reduces the implementation of the checker.

4.3 Why excluded computation inputs lose no theorems

The current certified-zero draft already proves

$$\zeta = I(2, 1; v.0) = 0.$$

Its direct successor computation has nonempty predecessor and initializer. For a computation leaf with a forbidden literal-zero initializer, or a successor predecessor literally zero, replace exactly those positions simultaneously by a fresh variable w, obtaining a motive $E(w)$. Then $E(\zeta)$ is an allowed direct computation. Transport through $\zeta = 0$ derives $E(0)$, the desired original leaf. Affected leaves use three native inference nodes: two computations and one transport. Passive zeros inside the fixed update are left unchanged. This inherited normalization is independent of the new uniform-premise argument, and is included to make the arithmetic dispatch complete.

5 Opening, arithmetic generation and the exact scope boundary

This section restates the inherited opening and generated-computation arguments [3, 2] for the selected instruction format. It explains the semantic facts used in the new trace proof; it does not charge an external validator to the untrusted producer.

5.1 An explicit decoder for admitted numerical coordinates

At a meaningful numerical position with p protected slots, the depth is $3+2p$. Nonnegative coordinates in $[0, 3)$ denote zero. For $0 \le j < p$, coordinates in $[3 + 2j, 5 + 2j)$ denote slot j, starting with the innermost slot. Thus a canonical integer slot has the two spellings $3 + 2j, 4 + 2j$. This interval convention also covers noninteger JSON coordinates. Negative admitted numbers are distinct rigid parameters. Python's booleans act as coordinate values zero and one; both coordinates decode to numerical zero. Values that fail the comparison, including nonfinite coordinates, can only survive at meaningful numerical positions by being replaced by a validated image; they are not extra numerical constants in the object language.

At a formula position scalar padding is the chosen constant formula reading. The numerical reading of a unary successor's child uses two additional zero slots. An iterator's counter and initializer use one additional zero slot; its update uses one state slot. A paired matrix has the two slots y, x, and a closure matrix has slot y. These are exactly the environments described by the array table. Surplus numerical arrays denote zero and surplus formula arrays denote truth, independently of their annotations.

Lemma 5.1 (Semantic opening)**.** *For every successful opening at a meaningful numerical root $3 + 2p$ or formula root $2p$, its decoded result is capture-free filling of the distinguished numerical hole in the decoded template. Protected slots and admitted rigid parameters retain their meaning. A successful no-image check preserves the meaning of a closed expression.*

Proof. Induct on the finite traversed tree. The scalar interval comparison protects exactly the displayed slots and admitted names. Outside that interval, a relevant numerical occurrence is filled by the image. The image is checked at numerical root depth three: it therefore has no free positive slot, and its free numerical parameters are rigid negative names. Copying it under additional slots captures none of these names. Its own internal binders retain their relative indices.

At a meaningful unary, binary or ternary array, the depth increments $4, 3, 2$ create exactly the environments in the decoding table. An ignored child has no effect on the mathematical meaning. Traversal of an ignored annotation can fail and reject a certificate; if it succeeds, the relevant constant meaning remains unchanged. In particular, negative child depths inside a surplus array do not require a fictitious numerical interpretation of those ignored children. These cases prove both filling and validation. Structural equality of successfully checked raw results consequently implies equality of their decoded meanings. □

No claim is needed that every raw tree is idempotent under every later revalidation. Surplus arrays can have odd validation behavior in ignored annotations while still having a constant meaning. The lemma is stated only at meaningful sorted positions and for successful calls.

5.2 Why fresh names cannot escape into a retained conclusion

The global floor only decreases. When its old value is K, its next allocated integer label is $K - 1$. Earlier relevant goals and context entries have passed no-image validation, so they contain no relevant numerical occurrence below their then-current floor. They therefore contain none of the names newly admitted in $[K - 1, K)$. An occurrence inside an ignored annotation is not a free variable of the decoded formula.

For integral canonical data there is one new label. For arbitrary finite raw numerical data, finitely many additional fractional names may lie in the interval. They are distinct fresh parameters, not aliases of its integer endpoint. Whenever a generated assumption uses them, the zero-instance discharge replaces every one of them simultaneously by zero. The retained context and conclusion are fixed.

5.3 The generated computation equation

With empty major and nonempty initializer u, the raw assumption is

$$H = \begin{cases} [\,[s, u, t], u\,], & s \text{ falsy}, \\ [\,[[s], u, t],\ i(t, [s, u, t], 3)\,], & s \text{ truthy}. \end{cases}$$

It is always a binary formula array. Its final image may be zero or the fresh parameter, as selected by the common tail of the code.

Lemma 5.2 (Zero-instance computation)**.** *If the generated expression and the subsequent checks succeed, the decoded assumption after simultaneous zero substitution for the newly admitted names is an instance of I0 or IS.*

Proof. In the falsy predecessor case, every successfully checked meaningful numerical predecessor is a zero representation. Empty arrays, zero numbers and false denote zero; a null, string or mapping in a meaningful numerical position fails. Both endpoint readings of the initializer agree after the new names and unprotected holes become zero. This is I0.

In the truthy case the left counter is explicitly a successor. If the update depends on its state in a relevant position, opening t at root three inserts the source $[s, u, t]$. The insertion first validates that entire source at numerical root three. In particular it revisits its update under precisely one state slot; an extra escaping update coordinate fails. It also checks counter and initializer with their prescribed dummy zero slot. The source therefore has exactly the state interpretation needed by the recursive value on the right.

Additional dummy slots inside the explicitly constructed successor do not change the decoded predecessor: closed relevant positive coordinates denote those zero slots, and rigid parameters are unchanged. Semantic opening now identifies the right endpoint with the update applied to that recursive predecessor value. This is IS. If the update does not use its state, its zero-normalized decoded value is independent of the unused image on both sides, which again gives IS. Checks performed only inside ignored annotations can reject more data but cannot change this successful decoded equation. □

The zero-instance conclusion is all that the continuation requires. The admissible zero-instance rule simultaneously replaces every newly admitted name by zero and then discharges the generated assumption. No assertion that every raw open assumption is uniformly true is needed. In particular, if the raw initializer is coordinate 3, the selected format opens its separate endpoint at zero: the tail test `len(a)%2 == u` is then false.

5.4 The paired slots and the induction restriction

For a unary D goal, `i(*a,t,2)` opens its witness slot at the numerical witness t and retains the index slot. The final goal opening with `[k]` supplies the successor of the fresh index. The local hypothesis is the arbitrary-index instance when $u = 0$ and the opened matrix has even arity; otherwise it is the zero instance. The only nontrivial even-arity formula constructor is the binary equation. Other even arities denote truth. Thus no induction on a nontrivial compound formula is silently admitted.

For a unary major $[B]$, triplication gives $[B, B, B]$. Opening this at the fresh name leaves its index bound and replaces the existential witness slot. The continuation is exactly DE. For a nonunary nonempty major, the export has at least six fields and denotes a harmless truth. These inherited slot calculations continue to hold when the verified empty major replaces the optional-premise opcode.

6 A persistent truth removes the optional-major branch

Let $\mathcal{A}$ be the explicitly stated native theory with numerical constructors $0, S, I$, equations, the paired binder D, and auxiliary closure Q. Write $\top_* = (0 = 0)$ and $\bot_* = (0 = S0)$. The native theory proves $\top_*$ and has elimination from $\bot_*$. Its full-iPRA interpretation is left unchanged.

6.1 The persistent truth and the compulsory premise

Retain the inherited numerical decoder and canonical formula decodings. The empty array and arrays of arity at least four read as $\top_*$. Unary, binary, and ternary arrays retain their D, equation, and Q

readings. As in the inherited representation, scalar padding at formula positions may be assigned the harmless truth reading. The proof below shows that a raw numeric scalar never succeeds as a checking goal.

The initial context is the one-element list containing the empty array. Thus it contains one proved truth. Every non-hypothesis packet now checks its submitted major. The canonical absent major is the empty array; its compulsory proof is an immediate hypothesis use. In a paired-introduction packet, the case/progress marker is moved to the initializer field: $u = []$ requests cases and $u = 0$ requests progress where the opened matrix is a literal equation. A nonempty initializer still selects the inherited computation branch.

The point is not merely to omit a guard. Python identifies numeric zero with its false sentinel, so an arbitrary unchecked scalar major could accidentally match the closure branch. A persistent truth premise and the invariant below make that scalar case unreachable on every successful checking trace. No input type promise or uncounted wrapper is assumed.

6.2 The root-shape invariant

Call an expression *array rooted* when its outermost Python value is a list. The distinction concerns the outer constructor only. It places no restriction on the arities or contents of that list.

Lemma 6.1 (Root preservation)**.** *Whenever the explicit-list opener succeeds, an array input has an array output and a numeric scalar input has a numeric scalar output at formula root depth. Other raw JSON values fail its scalar comparison. The transport assignment preserves the root class of the current goal.*

Proof. The array clause returns a list comprehension. At formula root depth zero, a numeric scalar is retained or replaced by scalar zero; no numerical image is inserted at that root. Nonnumeric nonarrays cannot pass the scalar ordering comparison.

For a successful transport match, the target goal equals the opening of its supplied template at one endpoint. Both endpoint openings have the same root class as that template. Replacing the target by the other opening therefore preserves the root class. A failed match leaves the goal unchanged. The compact profile's false-valued source fallback likewise leaves it unchanged. □

Lemma 6.2 (Successful-trace invariant)**.** *Suppose all entries of the incoming context are array rooted. If a finite call to the explicit-list checker succeeds, its original goal is array rooted, and every newly appended hypothesis in that successful trace is array rooted. Every compulsory major in the trace is therefore array rooted.*

Proof. Induct on the finite successful checking tree. If hypothesis closure succeeds, the opened goal equals an array-rooted context entry. Root preservation implies that the original goal is array rooted.

Otherwise the compulsory recursive call on the major succeeds first. Its incoming context has only array-rooted entries, so the induction hypothesis implies that the raw major m is a list. If m is nonempty, its export $m * 3$ is a list. If it is empty and computation is selected, the generated hypothesis is an explicit binary list. If it is empty and introduction is selected, the hypothesis and updated goal are the same opened matrix. A scalar matrix would fail the subsequent length operation because the empty major cannot bypass that test. Hence in a successful introduction this hypothesis is again array rooted. The original introduction goal must itself have been an array: its initial opener rejects nonnumeric nonarrays, and a numeric scalar cannot be starred into the introduction call.

Consequently the last recursive call has an array-rooted appended hypothesis and an incoming context consisting entirely of arrays. Its induction hypothesis implies that its goal is array rooted. In the major and computation branches, root preservation, including transport preservation, then implies that the original goal was array rooted. The introduction case was already checked. This establishes both claims simultaneously. □

The lemma applies at the public entry point because the sole initial hypothesis is the empty array. Numeric zero and false can never be accepted majors, even if supplied adversarially in a packet. The argument concerns successful finite traces; it is not a probability claim derived from negative tests.

6.3 Opening and freshness

The inherited opening operation traverses arrays at the same depths as before. The numerical cases, capture-free insertion, formula padding, and array interpretation are unchanged. Before a new floor is allocated, the retained context and current relevant goal have passed the no-image check. They therefore contain no newly admitted relevant numerical parameter. If a finite input contains fractional negative names, all additional names in the new interval are fresh. The zero-substitution argument below substitutes zero for each of them, not merely for the integer endpoint.

Theorem 6.3 (Soundness of the mandatory-major presentation)**.** *Every finite successful run of the explicit-list checker has a decoded conclusion derivable in $\mathcal{A}$ from its decoded retained assumptions. In particular, every closed conclusion accepted from the initial context is a theorem of $\mathcal{A}$.*

Proof. Apply Lemma 6.2 and induct on the finite successful checking trace. Hypothesis closure uses semantic opening. Every other packet proves an array-rooted major first. There are exactly two possibilities.

Nonempty array major. The inherited major argument applies. A match with the binary array $[s, u]$ is equality transport. A match with $[t, t, t]$ when s is false-valued is closure specialization from its zero instance. A mismatch retains the old goal. For the compact profile, if its Boolean expression declines to replace a goal by a false-valued source representation, the unchanged goal is still checked, which only strengthens the obligation.

Tripling a unary major yields the canonical three-copy closure of its matrix, and the continuation is paired elimination with the fresh witness. Tripling an array with at least two entries yields an array with at least six entries, hence a proved truth alias. The inherited opening and freshness lemmas justify the continuation. The numeric false sentinel cannot match this nonempty array major.

Empty array major. Its compulsory premise is $\top_*$. An empty array cannot equal the binary equality, the nonempty triple closure, or the numeric false sentinel. Thus the transport test leaves the original goal unchanged. The empty export selects computation or introduction.

For introduction, exact unary unpacking handles a genuine D goal. The witness is opened, the index coordinate retained, and the continuation checks the successor instance. When $u = 0$, the test $\mathrm{len}(a) \bmod 2 = u$ retains the arbitrary-index hypothesis only for an even-arity opened body. Among nontrivial formula constructors this is precisely a literal equation. Empty and larger even arities mean truth, where extra progress instances are admissible. Unary D and ternary Q bodies receive the zero-instance hypothesis, as required by cases. The marker $u = []$ always selects cases. Other successfully used false-valued numeric markers have the same numerical-zero role. A scalar opened

body fails its length operation. The undersupplied empty-goal case already has conclusion $\top_*$; exact unpacking rejects a binary goal and arrays with more than one argument. In particular no nontrivial compound induction rule is admitted.

For computation, the generated hypothesis is a binary equation. Its all-zero fresh-name instance is the inherited valid computation instance, including the certified-zero repair where needed. The test involving u can select an open fresh instance instead of its zero instance. The open equation need not be universally true. Since the old context and goal are fresh for the newly admitted names, substitute zero for all of them in the continuation proof $\Gamma, H(\vec{b}) \vdash A$ and cut in the proved $H(\vec{0})$. This gives $\Gamma \vdash A$. The same argument covers incidental extra parameters in a fractional interval.

These are all successful cases by the root-shape invariant. The initial empty-array assumption decodes to the theorem $\top_*$, and ordinary assumption cut discharges it at the public entry point. □

6.4 A second, semantic check

One may instead assign $\bot_*$ to every numeric scalar at a formula position, leaving its numerical meaning and all array clauses unchanged. The opening lemma still holds because scalar padding is retained or replaced by scalar zero, which now has the same falsity reading. Under this optional completion, a successful numeric-major call would already supply native contradiction, and explosion would justify the parent conclusion irrespective of the false-sentinel match. Nonempty and empty array majors have the soundness proof just given.

Thus bottom completion gives a second sufficient semantic argument. The successful-trace invariant is stronger: numeric majors are actually unreachable, so changing padding to falsity is unnecessary. Neither argument asserts that raw padding is part of the native object language or that every raw expression has an internally provable equivalence with its assigned decoder meaning.

Theorem 6.4 (Canonical native consequences are preserved)**.** *Every finite native derivation has a certificate in the mandatory-major presentation. On canonical native contexts and conclusions the new checker presents the same consequence relation as $\mathcal{A}$. Its ordinary realization is equiconsistent and bi-interpretable with full first-order iPRA through the inherited whole-domain interpretations.*

Proof. First apply the inherited finite repair of excluded computation leaves using the independently proved nonempty zero initializer. Take the canonical backward packet certificate of the resulting derivation. Keep every actual major and all canonical formulas. Replace each absent-major marker by the empty array. In an introduction copy the old case/progress choice into u; in a computation retain the nonempty initializer. The added initial truth remains in every retained context.

Reserialize the resulting finite proof in the new context, pruning any subtree whose goal already closes as a hypothesis, and allocating names for the remaining proof. This is not an assertion that every arbitrary historical raw packet stream survives unchanged. Each mandatory empty-major call is now a hypothesis leaf and consumes no instruction packet. All actual majors, computation instances, and introduction obligations have their original native meanings. Induction on the finite native derivation proves successful checking in the unbounded-resource mathematical reading. The transformation adds checker calls at absent-major sites but adds no instruction packets; it is not claimed to reduce runtime or serialized proof size.

Soundness and this transformation give exactly the native consequence relation on canonical expressions. The identity on native syntax transfers the inherited full numerical and formula interpretation composites. The numerical domain remains the whole domain, and the composite isomorphism has graph $x = y$. The raw completion is an implementation semantics for certificate data, not an additional interpreted object sort or a separate raw-syntax bi-interpretability claim. □

The new theorem permits a uniform mandatory-major instruction by proving that its unwanted scalar branch cannot succeed. The arithmetic basis, logical reconstruction, and full-iPRA bridge remain inherited. This is a structural proof-presentation reduction, not an absolute minimality theorem or a new arithmetic foundation. The written trace induction is not a proof-assistant certificate.

7 Validation, counterchecks and reproducibility

The exact delivered primary and compact checkers passed 184 full-process expectations: 46 cases, two programs, and ordinary plus optimized Python. Each mode/profile combination includes 20 accepted and 26 rejected certificates. The named producer separately regenerates and replays the 31 inherited core cases. The cases include genuine atomic induction, conditional-closure specialization, nonvacuous paired elimination, nested binders, contextual induction, escaping variables, malformed data, nonempty-zero repair, and the fresh-computation zero-instance boundary. Finite tests supplement the trace and interpretation proofs; they do not establish universal soundness on their own.

An independent review tested a closely related explicit-list candidate before selection of the ordinary conditional: 22,248 negative checks and nine complete CLI controls had no mismatches. Its exact source hash and code are retained as `reference/reviewed_boolean_checker.py`. This evidence is not mislabeled as a separate 22,248-case run of the final conditional file. The inherited arithmetic bridge was also reviewed for fixed-parameter induction and circular dependencies. The pairing appendix abbreviates routine order/semiring derivations; none of these written proofs is a Lean or Rocq formalization.

7.1 A mutation that really proves a false equation

The following raw certificate requests $1 = 0$:

```
[
  [[], [0], [0], 0],
  [false, [[[0]], [0], 0], [0], [3, 0]],
  [[0], 0]
]
```

Both delivered checkers reject it. Delete only the unconditional `c(m, g)` line: both mutants accept the false conclusion, in ordinary and optimized Python. The false major now collides with the false sentinel in the combined matcher and rewrites the goal without an equality proof. Thus the omitted guard has been replaced by a real proof obligation, not by trusting the input. These four mutation checks are part of the default verification command.

7.2 Canonical completeness has an explicit boundary

Sound decoding of an accepted raw expression does not imply completeness for every arbitrary raw spelling of a true formula. The raw expression `[0]` has scalar padding as its paired body. Its short introduction attempt rejects at the matrix length check. Under the optional bottom decoder its meaning is $D(\bot)$, which is true, but this is not its canonical spelling. The canonical $D(0 = S0)$, encoded by `[[0,[0]]]`, has the expected accepted case proof. The general completeness transformation starts with a native derivation, regenerates its certificate in the new context, and may prune truth subproofs. It does not promise to preserve every historical raw packet stream.

The selected programs use explicit finite-tree list recognition and do not use the generic iterable trick in some comparison profiles. As with ordinary recursive proof checkers, mathematical completeness assumes adequate resources; finite CPython recursion and memory limits can reject a sufficiently large valid certificate.

7.3 Files and commands

The standalone `ipra_uniform_premise.tex` contains the main text, new proof, listings and all attributed mathematical appendices. It can be compiled without the modular authoring sources. The package also retains `report.tex`, the exact attributed source sections, both standalone checkers, a readable expansion, fixtures, untrusted producer, the source corpus and evidence files.

From a fresh extraction:

```
python3 reproduce.py
python3 code/checker.py < examples/certified_zero.json
python3 code/checker.py < examples/scalar_false_sentinel_attack.json
python3 build.py
```

The third command must reject with nonzero exit status. The reproduction command checks file hashes, remeasures all 443 frozen sources and the delivered programs, and runs the selected CLI and mutation controls. No network access is required. It never executes historical comparison source during the measurement pass. The build uses ordinary pdfLaTeX without shell escape. Gzip files of the two checkers are included for the informational compression measurement.

8 Assessment of the requested breakthrough

The new mechanism is a proved reduction in the complete proof presentation: every instruction has a major, the trivial one is already proved, and successful proof checking enforces the required constructor class. This is an advance over the pinned implementations and gives a small source-byte record within that collection.

It leaves the arithmetic and logical basis intact. It does not show that the native theory is absolutely simplest, that the kernel is optimal under a fixed universal description language, or that one implementation beats all current source-byte, token and AST minima. No major new foundation meeting the stronger part of the request was established in this investigation. Calling the measured reduction such a breakthrough would overstate the result.

Attribution for the complete inherited bridge

The following five sections are reproduced unchanged from the reference files in `ipra_certified_zero_2026-11-10.zip` at the pinned commit. They retain that report's historical attributions. Their original paths and exact hashes are recorded in `evidence/inherited_provenance.json`; the repository's MIT No Attribution license is included. They are inherited mathematics, not new results of this report. Phrases such as "the new choice" within these source sections refer to their original development.

The first appendix uses the older clause notation $\langle B(0); y.B\rangle$ for the conditional closure now written $\mathsf{Q}_y B$. Its ordinary assumptions are constant clauses with a fixed computation theorem as antecedent. Hyp and QE in the main text implement the same uses. The unused-witness D construction converts between clauses and the public Q form; the stated structural admissibility then gives the same native consequences. No external logic oracle is silently supplied to the code.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n,a;u.h), \qquad I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n,a,b) = I(n,a;u.b), \qquad Z(n) = C(n,1,0), \qquad a+b = I(b,a;u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$
$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `76a2651601d17b34d46753f189ea026a21ce8d29`. https://github.com/VladimirReshetnikov/ProveIt/tree/76a2651601d17b34d46753f189ea026a21ce8d29/docs/drafts. All archive citations below refer to this immutable collection.

[2] *A certified zero removes a computation opcode.* `ipra_certified_zero_2026-11-10.zip`, in [1]. Source of the smallest current byte/token profiles, initializer dispatch, certified-zero normalization, the direct predecessor computation generator, the attributed bridge appendices, and the baseline fixtures.

[3] *An Arity Calculus for iPRA.* `ipra_arity_calculus_2026-11-10.zip`, in [1]. Source of the unary paired binder, triplicate closure, numerical root depth three and uniform arity opener.

[4] *Uniform Predecessor Certificates for Intuitionistic Primitive Recursive Arithmetic.* `ipra_uniform_predecessor_2026-11-10.zip`, in [1]. Source of the predecessor computation instruction and earlier bridge presentation.

[5] *Affine supply for intuitionistic primitive recursive arithmetic.* `ipra_affine_supply_2026-11-10.zip`, in [1]. Source of a second checker attaining the current AST minimum.

[6] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic.* Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf. Used for the conventional full first-order iPRA/atomic-induction interface; it is not a source for the new implementation theorem.

[7] Inherited scalar predecessor development, attributed in `reference/scalar_predecessor.tex` of [2]. Reproduced with its source hash in this package.

[8] Inherited forward equality-reflection development, attributed in `reference/equality_reflection.tex` of [2]. Reproduced with its source hash in this package.

[9] Inherited progress-closure arithmetic and primitive-recursion compiler, attributed in `reference/primitive_recursion.tex` of [2]. Reproduced with its source hash in this package.